

Protocolo Carrasco

Continuação a partir do exercício 31 · 247 exercícios detalhados · versão 3 (26/09/2026)

Onde você está

Exercício 30 de uma trilha de mais de 300. Dos 30 feitos, **10 tinham bug ou requisito errado** (3, 4, 8, 13, 20, 22, 23, 24, 25, 30) — um em cada três. Nenhum tinha método extraído, nenhum tinha teste, nenhum estava no Git. Os erros são de base: divisão inteira, Integer com ==, switch sem break, fórmula errada, caso de borda sem tratar. Nada disso é grave no exercício 30. Seria eliminatório num teste técnico. Esta lista existe para que não seja.

Regras (não negociáveis)

Regra	Na prática
1. IA não escreve código	Você escreve. IA só revisa DEPOIS de pronto e testado. Seus projetos anteriores foram feitos com IA; esta lista é para provar que você programa sem ela.
2. Definição de pronto	Compila, passa nos testes (a partir do J), está no GitHub e você explica em voz alta sem ler. Faltou um → não está pronto.
3. Travou	30 min: documentação oficial (em inglês). 60 min: veja a solução, feche, apague seu código e refaça do zero no dia seguinte.
4. Sem pular	Desafio de módulo que não cumpriu o critério de pronto = refaz o desafio antes de avançar.
5. Revisão semanal	Todo domingo: refaça 3 exercícios antigos sorteados, do zero, sem olhar o que fez antes.
6. SQL todo dia	Trilha S em paralelo, 40 min por dia, começando amanhã.
7. Atenção em vermelho	Todo “Atenção” é pegadinha de entrevista. Você tem que saber explicar cada um.

Lacunas: onde cada uma é coberta

Lacuna	Por que trava a vaga	Onde
Maven	Sem isso você não builda nenhum projeto Spring	31-41
JUnit / testes	Todo projeto Spring tem teste; pergunta padrão de entrevista	42-51, 190-195
Git / GitHub	Portfólio é como a empresa te encontra	31-41
HTTP / REST / JSON	É o que você vai construir no Spring	215-224
Stack trace / debug	Metade do trabalho é achar bug que você não escreveu	31-41 (e em todo o resto)
SQL	Todo backend fala com banco; teste técnico quase sempre tem SQL	S1-S38
JDBC e transação	É o que o Spring Data e o @Transactional fazem por baixo	204-214
BigDecimal	double para dinheiro é bug em sistema real	74-97

Lacuna	Por que trava a vaga	Onde
Camadas e injeção de dependência	Controller → Service → Repository é a estrutura do Spring	170-189
Mockito	É como se testa Service no Spring	190-195
Logging	Em produção não existe System.out	170-189
Big-O e estruturas de dados	Cai em teste técnico e em entrevista	52-73, 114-138
Concorrência básica	Servidor web atende várias requisições ao mesmo tempo	196-200
Docker	Subir banco e app localmente é rotina	201-203
Terminal	Servidor não tem IDE	31-41
Algoritmos e live coding	Etapa comum em processo de júnior	E1-E4

Ordem

C1-C10 Correções obrigatórias em 1-30 → 31-41 Ferramentas → 42-51 JUnit 5 na prática → 52-73 Strings e Arrays → 74-97 POO → 98-113 Exceptions → 114-138 Collections → 139-153 Generics, Enum e Date/Time → 154-169 Lambda, Streams e Optional → 170-189 Java Core e design antes do Spring → 190-195 Testes → 196-200 Concorrência básica → 201-203 Docker mínimo → 204-214 JDBC → 215-224 HTTP, REST e JSON → 225 Projeto final. **Em paralelo desde amanhã:** S1-S38 (SQL) e E1-E4 (entrevista). JDBC só depois de S1-S30.

C1-C10 | Correções obrigatórias em 1-30

Bugs reais encontrados rodando o seu código. Faça antes do 31 — cada um é pegadinha clássica de entrevista.

☐ **C1. Ex. 30 — média com divisão inteira**

A média de 1 e 2 sai 1. Digitar 0 como primeiro número estoura `ArithmeticException: / by zero`. Use `double` para a média e trate o caso “nenhum número digitado”.

Exemplo: 1, 2, 0 → média: 1.50. 0 → Nenhum número informado.

Atenção: soma / contador com dois `int` sempre trunca. Converter depois `((double)(soma / contador))` não resolve — converta antes.

☐ **C2. Ex. 3 e 4 — mesma divisão inteira**

7 / 2 imprime 3. O ex. 4 só “funciona” porque $14+41+23 = 78$ é múltiplo de 3. Troque os valores para 1, 2, 2 e veja quebrar. No ex. 3, trate divisão por zero.

Exemplo: 7 e 2 → `DIVISÃO = 3.5`. 7 e 0 → Não é possível dividir por zero.

☐ **C3. Ex. 8 — fórmula errada**

Área do círculo é $\pi \cdot r^2$, não $2 \cdot \pi \cdot r^2$ ($2 \cdot \pi \cdot r$ é o perímetro). Raio 1 deu 6,28; o certo é 3,14.

Atenção: Use `Math.pow(raio, 2)` ou `raio * raio`. Erro fórmula é erro de requisito, não de Java — leia o enunciado.

☐ **C4. Ex. 13 — reescrever do zero**

Lê 4 números em vez de 2, não trata “o segundo é maior”, o `printf` usa `2.%n` em vez de `%d`, e compara `Integer` com `==`. Com 500 e 500 não imprime “iguais”.

Exemplo: 5 e 9 → 9 é maior que 5. 500 e 500 → Números iguais.

Atenção: `Integer` guarda cache de -128 a 127, então `==` funciona para números pequenos e falha para grandes. Use `int` ou `equals()`. Isso cai em entrevista.

☐ **C5. Ex. 20 — isósceles incompleto**

3, 4, 3 sai “Escaleno” porque faltou o caso `a == c`. Troque o ternário aninhado por um método `classificar(a, b, c)` com `if/else`.

Atenção: Verifique equilátero primeiro; isósceles = dois lados iguais (qualquer par). Escreva as 4 saídas possíveis antes de codar.

☐ **C6. Ex. 25 — fall-through e menu que não é menu**

A opção 0 imprime “Saindo” e depois “Operação inválida” (faltou `break`). Também pede dois números antes de sair. Um menu de verdade repete até o usuário escolher sair.

Exemplo: Loop `while (opcao != 0)`; ler a opção primeiro; só pedir números se a opção for 1-4.

Atenção: Reescreva com `switch` de seta (`case 1 -> ...`), que não tem fall-through.

☐ **C7. Ex. 24 — mês inválido**

Mês 13 estoura `ArrayIndexOutOfBoundsException`. Valide 1-12 e lance

`IllegalArgumentException("Mês inválido: 13")`. Faça 3 versões: lista, `switch expression` e `java.time.Month`.

Exemplo: `Month.of(3).getDisplayName(TextStyle.FULL, Locale.of("pt", "BR"))` → março.

☐ **C8. Ex. 22 — requisito mal interpretado**

Aumento salarial é $\text{salário} \times (1 + \text{percentual}/100)$. O usuário informa salário e percentual; o programa mostra valor do aumento e novo salário.

Exemplo: 2000 e 10% → Aumento: 200.00 | Novo salário: 2200.00.

☐ **C9. Ex. 23 — calculadora que não aceita decimal**

Lê com `nextInt()` para uma variável `double`: digitar 2.5 quebra. Use `nextDouble()` e trate divisão por zero (com `double`, 5/0 vira `Infinity` sem erro nenhum).

Atenção: Descubra por que `nextDouble()` pode exigir vírgula no seu computador (Locale). Use `new Scanner(System.in).useLocale(Locale.US)` se quiser ponto.

☐ **C10. Tirar a lógica do main (vale para todos)**

Para cada exercício de 11 a 30, crie um método que recebe parâmetros e retorna o resultado. O `main` só lê entrada e imprime. Exemplos: `boolean isBissexto(int ano)`, `String classificarNota(double nota)`, `double media(List<Integer> numeros)`.

Atenção: Sem método não existe teste unitário. Saiba também escrever o clássico `public static void main(String[] args)` — o `static void main()` sem `public` só compila no Java 25+ e muito código de empresa ainda roda em 17 ou 21.

31-41 | Ferramentas: terminal, Git, Maven, stack trace e debugger **NOVO**

O que separa “sei Java” de “consigo trabalhar num projeto”. Tudo daqui em diante vive num projeto Maven versionado no GitHub.

☐ **31. Compilar e rodar sem IDE**

Crie `01a.java` num editor simples. Compile com `javac`, rode com `java`, veja o `.class` gerado. Depois coloque a classe num package `br.com.kawa` e faça funcionar de novo.

Exemplo: `javac -d out src/br/com/kawa/01a.java` e `java -cp out br.com.kawa.01a`

Atenção: Entenda por que o package precisa bater com a pasta e o que é `classpath`. Toda mensagem “Could not find or load main class” vem daqui.

☐ **32. Terminal básico**

Pelo terminal: crie a estrutura de pastas de um projeto, copie, mova e apague arquivos, e ache todos os exercícios que usam `Scanner`.

Exemplo: `mkdir -p, cd, ls -la, cp, mv, rm, cat, grep -rn "Scanner" ., grep ... | wc -l`

Atenção: Servidor não tem IDE. Logs, deploy e Docker são tudo terminal.

☐ **33. Git local**

`git init` na pasta dos exercícios. Crie um `.gitignore` (`target/`, `out/`, `.idea/`, `*.class`). Faça um commit por correção C1-C10.

Exemplo: `git status, git add, git commit -m "corrige divisao inteira no ex30", git log --oneline, git diff`

Atenção: Commit pequeno com mensagem que diz o QUÊ mudou. “update” e “ajustes” não servem.

☐ **34. GitHub e README**

Crie um repositório público, faça `git push`, e escreva um README com: o que é o repositório, como compilar, como rodar os testes.

Atenção: Esse repositório é o seu portfólio. Recrutador abre o GitHub antes do currículo.

☐ **35. Branch, Pull Request e merge**

Crie a branch `strings-arrays`, faça os próximos exercícios nela, abra um Pull Request no GitHub, revise o próprio diff e faça o merge na `main`.

Exemplo: `git switch -c strings-arrays, git push -u origin strings-arrays`

Atenção: Em empresa ninguém commita direto na `main`. PR é o fluxo diário.

☐ **36. Resolver conflito**

Em duas branches diferentes, altere a mesma linha do mesmo arquivo. Faça merge e resolva o conflito na mão (sem aceitar “tudo meu” ou “tudo deles” às cegas).

Atenção: Entenda os marcadores <<<<<<, ===== e >>>>>>.

☐ **37. Projeto Maven do zero**

Crie um projeto Maven (pela IDE ou `mvn archetype:generate`). Configure Java 21 no `pom.xml` com `maven.compiler.release`. Rode `mvn compile`, `mvn test`, `mvn package` e veja o que aparece em `target/`.

Atenção: Saiba explicar `groupId`, `artifactId`, `version`, `src/main/java` vs `src/test/java` e o ciclo de vida (`compile` → `test` → `package` → `install`).

☐ **38. Dependências Maven**

Adicione `junit-jupiter` com `<scope>test</scope>`. Rode `mvn dependency:tree`. Ache a pasta `~/.m2/repository` e veja o jar baixado.

Atenção: Diferença entre `scope compile`, `test` e `provided`. Um projeto Spring Boot é basicamente um pom com dependências — se isso é mágica, o Spring vira mágica também.

☐ **39. Ler stack trace**

Escreva `main` → `processar` → `calcular` onde `calcular` lança exceção. Provoque 5 tipos: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`, `ClassCastException`, `ArithmeticException`. Para cada uma anote: tipo, mensagem, arquivo:linha de origem e quem chamou.

Exemplo: A primeira linha `at ...` é onde a exceção nasceu; as de baixo são quem chamou. Procure a primeira linha que é do SEU código.

Atenção: Com exceção encadeada, leia também o bloco `Caused by:` — normalmente o problema real está nele.

☐ **40. Debugger da IDE**

Coloque um breakpoint no ex. 30 (versão com bug), rode em modo debug e acompanhe soma, contador e media com `step over` / `step into` / `step out`. Depois use um breakpoint condicional no loop de 1 a 100 que só para em `i == 50`.

Atenção: Use “Evaluate Expression” para testar soma / (double) contador sem mudar o código.

☐ **41. Caçar bug com o debugger**

Escreva `int fatorial(int n)` com loop. `fatorial(12)` está certo, `fatorial(13)` dá errado e `fatorial(34)` dá 0. Use o debugger para ver o momento exato em que o valor “vira”. Corrija com `long`, depois descubra o limite do `long` e resolva com `BigInteger`.

Atenção: Overflow não lança exceção em Java. Conheça `Math.multiplyExact`, que lança.

42-51 | JUnit 5 na prática **novo**

Aqui o teste deixa de ser opcional. Use os métodos que você extraiu no C10.

☐ **42. Primeiro teste**

Mova `isBissesto` para `src/main/java/.../Datas.java` e crie `DatasTest` em `src/test/java`. Teste 2000, 1900, 2024 e 2023. Rode com `mvn test`.

Exemplo: `assertTrue(Datas.isBissesto(2000)); assertFalse(Datas.isBissesto(1900));`

Atenção: Teste que nunca falhou não prova nada. Quebre o método de propósito e veja o teste ficar vermelho.

- ☐ **43. Estrutura e nomes**
Reescreva os testes no formato Arrange / Act / Assert e com nomes que descrevem o comportamento. Use `@DisplayName`.
Exemplo: `deveRetornarFalse_quandoAnoDivisivelPor100MasNaoPor400()`

- ☐ **44. Teste que encontra bug**
Crie `Triangulo.classificar(int a, int b, int c)` retornando um enum `TipoTriangulo` (EQUILATERO, ISOSCELES, ESCALENO). Lados que não formam triângulo lançam `IllegalArgumentException`. Escreva os testes ANTES de corrigir o C5.
Exemplo: (3, 4, 3) → ISOSCELES; (1, 2, 10) → exceção; (0, 0, 0) → exceção.

- ☐ **45. Comparar double**
Teste `Estatistica.media(List<Integer>)`: [1, 2] → 1.5; [10] → 10.0; lista vazia → `IllegalArgumentException`.
Exemplo: `assertEquals(1.5, media, 0.0001);`
Atenção: Nunca compare double com `==` ou `assertEquals` sem delta. Descubra por que `0.1 + 0.2 != 0.3`.

- ☐ **46. Testar exceção e mensagem**
Teste que `Meses.nome(13)` lança `IllegalArgumentException` com a mensagem "Mês inválido: 13".
Exemplo: `var ex = assertThrows(IllegalArgumentException.class, () -> Meses.nome(13)); assertEquals("Mês inválido: 13", ex.getMessage());`

- ☐ **47. Teste parametrizado**
Teste a calculadora com `@ParameterizedTest` e `@CsvSource` cobrindo as 4 operações em uma única função de teste. Divisão por zero fica num teste separado.
Exemplo: `@CsvSource({"2,+,3,5", "10,-,4,6", "3,*,4,12", "9,/,2,4.5"})`

- ☐ **48. Pegadinha do par e ímpar**
Teste `isImpar(int n)` com `@ValueSource` incluindo números negativos. Implemente como `n % 2 == 1` e veja -3 falhar. Corrija.
Atenção: Em Java, `-3 % 2` é -1. Use `n % 2 != 0`.

- ☐ **49. Ciclo de vida do teste**
Use `@BeforeEach` para criar o objeto testado e `@AfterEach` para imprimir "fim". Imprima o `hashCode()` da instância da classe de teste em cada método e prove que o JUnit cria uma instância nova por teste.
Atenção: Por isso testes não devem depender da ordem nem compartilhar estado.

- ☐ **50. AssertJ**
Adicione `AssertJ` e reescreva 5 testes com `assertThat`. Use `containsExactly`, `hasSize`, `isCloseTo`, `isInstanceOf`, `assertThatThrownBy`.
Atenção: É o estilo mais comum em projetos Spring reais.

- ☐ **51. Cobertura com JaCoCo**
Configure o plugin do JaCoCo no pom, rode `mvn test` e abra o relatório HTML. Ache um ramo do `Triangulo` sem teste e cubra.
Atenção: 100% de cobertura não significa código sem bug. Cobertura mostra o que NÃO foi testado.

52-73 | Strings e Arrays com TDD

Regra: escreva os testes primeiro, veja falhar, implemente. Cada exercício é um método estático numa classe (Textos, Arrays2) com sua classe de teste.

☐ 52. Contar caracteres

`int contarCaracteres(String texto, boolean contarEspacos).`

Exemplo: ("oi tudo", true) → 7; ("oi tudo", false) → 6.

Atenção: Decida o que fazer com null (retornar 0 ou lançar exceção), documento e teste.

☐ 53. Contar vogais

`int contarVogais(String texto)` aceitando maiúsculas e acentos.

Exemplo: "Ação" → 3; "PYTHON" → 1; "" → 0.

Atenção: Pesquise `java.text.Normalizer` para tirar acentos ("ã" → "a").

☐ 54. Contar consoantes

`int contarConsoantes(String texto)` ignorando números, espaços e pontuação.

Exemplo: "Java 21!" → 2.

Atenção: `Character.isLetter` ajuda. "ç" é consoante.

☐ 55. Inverter String

`String inverter(String s)` de duas formas: loop de trás para frente e `StringBuilder.reverse()`.

Exemplo: "backend" → "dnecab".

☐ 56. Palíndromo de verdade

`boolean isPalindromo(String s)` ignorando espaços, pontuação, maiúsculas e acentos.

Exemplo: "Socorram-me, subi no ônibus em Marrocos" → true; "Java" → false.

Atenção: Resolva com dois índices (início e fim) sem criar a string invertida.

☐ 57. Contar palavras

`int contarPalavras(String s).`

Exemplo: " oi tudo bem " → 3; "" → 0; " " → 0.

Atenção: `"".split(" ")` retorna um array de tamanho 1. Use `strip()` e `split("\\s+")` e teste os casos vazios.

☐ 58. Maior e menor palavra

`String maiorPalavra(String frase)` e `String menorPalavra(String frase)`. Em empate, retorna a primeira. Pontuação não conta.

Exemplo: "Eu estudo Java, todo dia!" → maior "estudo", menor "Eu".

☐ 59. Ocorrências de caractere

`int contarOcorrencias(String texto, char c, boolean ignorarCaixa).`

Exemplo: ("Banana", 'a', false) → 3; ("Banana", 'b', true) → 1.

☐ 60. Normalizar espaços

`String normalizarEspacos(String s)`: remove espaços do início/fim e troca sequências de espaços por um só. Faça também `removerTodosEspacos`.

Exemplo: " Kawã Viana " → "Kawã Viana".

Atenção: Diferença entre `trim()` e `strip()` (unicode).

-
- ☐ **61. Capitalizar nome**
String capitalizarNome(String nome) deixando “da, de, do, das, dos, e” minúsculos, exceto se forem a primeira palavra.
Exemplo: "kAWã viNICIUS da silva" → "Kawã Vinicius da Silva".
Atenção: Esse tipo de regra aparece de verdade em cadastro de sistema.
-
- ☐ **62. String é imutável**
Prove com código e testes: s.toUpperCase() sem atribuição não altera s; new String("a") == "a" é false; "a" == "a" é true; equals é true nos dois.
Atenção: Explique a String pool num comentário. Nunca compare String com ==.
-
- ☐ **63. Concatenação em loop**
Concatene 100.000 números numa String com += e depois com StringBuilder. Meça o tempo com System.nanoTime().
Atenção: Explique por que += em loop é lento (cria objetos novos a cada volta).
-
- ☐ **64. Validar CPF**
boolean isCpfValido(String cpf) aceitando com ou sem pontuação e calculando os dois dígitos verificadores.
Exemplo: "529.982.247-25" → true; "111.111.111-11" → false; "123" → false.
Atenção: Todos os dígitos iguais passam no cálculo mas são inválidos. Validação de documento é tarefa real de backend brasileiro.
-
- ☐ **65. Validar e-mail com regex**
boolean isEmailValido(String email) com regex simples: texto@texto.dominio.
Exemplo: "kawa@gmail.com" → true; "kawa@" → false; "kawa gmail.com" → false.
Atenção: Aprenda matches, Pattern e Matcher. Não existe regex perfeita de e-mail — no Spring você vai usar @Email.
-
- ☐ **66. Maior e menor do array**
MinMax encontrarMinMax(int[] numeros) retornando um record MinMax(int min, int max) numa única passada.
Exemplo: [4, -2, 9, 0] → MinMax[min=-2, max=9]. Array vazio → IllegalArgumentException.
Atenção: Não comece max com 0 — falha com array só de negativos.
-
- ☐ **67. Inverter array no lugar**
void inverter(int[] v) sem criar outro array, trocando as pontas com dois índices.
Exemplo: [1, 2, 3, 4, 5] → [5, 4, 3, 2, 1]. Teste tamanho par, ímpar, 1 e 0.
-
- ☐ **68. Remover duplicados mantendo ordem**
int[] removerDuplicados(int[] v) sem usar Collections (só arrays e loops).
Exemplo: [3, 1, 3, 2, 1] → [3, 1, 2].
Atenção: Depois de estudar Collections, refaça em uma linha com LinkedHashSet e compare.
-
- ☐ **69. Rotacionar array**
int[] rotacionarDireita(int[] v, int k).
Exemplo: ([1, 2, 3, 4, 5], 2) → [4, 5, 1, 2, 3]. k = 7 com 5 elementos equivale a k = 2.
Atenção: Use k % v.length. Cuidado com array vazio (divisão por zero no %).
-
- ☐ **70. Busca binária e Big-O**
Implemente busca linear e busca binária num array ordenado de 1.000.000 de números. Conte quantas comparações cada uma faz para achar o último elemento.
Atenção: Primeiro contato com Big-O: O(n) vs O(log n). Cai em entrevista.
-

- **71. Matriz**
Numa matriz 3×3: soma da diagonal principal, transposta e soma de cada linha.
Exemplo: [[1,2,3],[4,5,6],[7,8,9]] → diagonal 15; transposta [[1,4,7],[2,5,8],[3,6,9]].

- **72. Dois números que somam o alvo**
int[] doisQueSomam(int[] v, int alvo) retornando os índices. Faça com dois loops (O(n²)).
Exemplo: ([2, 7, 11, 15], 9) → [0, 1].
Atenção: Guarde este exercício: você refaz em O(n) com HashMap no módulo de Collections.

- **73. Anagrama**
boolean isAnagrama(String a, String b) ignorando espaços e maiúsculas.
Exemplo: ("Roma", "amor") → true; ("java", "jaca") → false.
Atenção: Duas soluções: ordenar os caracteres ou contar frequência num int[26].

74-97 | POO com um domínio de loja

Tudo aqui constrói o MESMO sistema: Pessoa, Cliente, Produto, Pedido, Estoque, Conta. Regras de negócio têm teste.

- **74. Classe Pessoa**
Crie Pessoa com nome e idade, instancie três pessoas no main e imprima os dados. Depois sobrescreva toString().
Exemplo: Pessoa[nome=Ana, idade=30]
Atenção: Sem toString() aparece algo como Pessoa@1b6d3586. Entenda por quê.

- **75. Encapsulamento de verdade**
Torne os atributos private. Crie getters. Crie setter só para o que pode mudar (idade sim, nome decida você). Valide no setter: idade entre 0 e 150, senão IllegalArgumentException.
Atenção: Getter e setter para tudo, sem validação, não é encapsulamento — é campo público com passos extras.

- **76. Construtor com validação**
Crie o construtor Pessoa(String nome, int idade) que rejeita nome nulo/vazio e idade inválida. O objeto nunca pode existir em estado inválido.
Exemplo: new Pessoa("", 20) → IllegalArgumentException("Nome é obrigatório").
Atenção: Teste os três casos inválidos com assertThrows.

- **77. Sobrecarga e this(...)**
Crie Pessoa(String nome) que chama this(nome, 0). Crie um método saudar() e uma sobrecarga saudar(String saudacao).
Atenção: Toda validação fica num construtor só; os outros delegam.

- **78. static vs instância**
Adicione um contador static de pessoas criadas e um id de instância que usa esse contador. Crie 3 pessoas e imprima ids e total.
Atenção: Explique num comentário por que campo static mutável é perigoso em servidor (várias requisições ao mesmo tempo).

- **79. Produto com BigDecimal**
Crie Produto(id, nome, preco) com preco em BigDecimal. Preço deve ser maior que zero.
Atenção: Crie com new BigDecimal("19.90") ou BigDecimal.valueOf(19.90), nunca new BigDecimal(19.90). Imprima os dois e veja a diferença.

- ☐ **80. Por que não double para dinheiro**
Some $0.1 + 0.2$ com `double` e com `BigDecimal`. Some R\$ 0,10 dez vezes nos dois tipos. Escreva um teste que prova que `double` erra.
Exemplo: `double` → 0.30000000000000004; `BigDecimal` → 0.3.
-
- ☐ **81. Desconto com arredondamento**
`BigDecimal Produto.precoComDesconto(BigDecimal percentual)` validando 0 a 100 e arredondando para 2 casas com `RoundingMode.HALF_EVEN`.
Exemplo: R\$ 19,90 com 15% → R\$ 16,92 (16,915 arredondado).
Atenção: `BigDecimal` com `equals` compara escala: 2.0 não é igual a 2.00. Use `compareTo` nos testes ou normalize a escala.
-
- ☐ **82. ContaBancaria**
`ContaBancaria(numero, titular)` com saldo começando em zero, sem setter de saldo. Métodos `depositar(valor)` e `sacar(valor)`. Valor ≤ 0 lança exceção. Saque maior que o saldo lança `IllegalStateException` (vira exceção própria no próximo módulo).
Exemplo: depositar 100, sacar 30 → saldo 70. sacar 100 → exceção, saldo continua 70.
Atenção: Teste que o saldo NÃO muda quando o saque falha.
-
- ☐ **83. Extrato**
Cada operação gera uma `Transacao(tipo, valor, dataHora)` guardada numa lista dentro da conta. `getExtrato()` retorna uma cópia imutável.
Atenção: Se você retornar a própria lista, quem chamar pode apagar o histórico da conta. Teste que `getExtrato().add(...)` lança `UnsupportedOperationException`.
-
- ☐ **84. Transferência**
`transferir(ContaBancaria destino, BigDecimal valor)`: saca de uma, deposita na outra. Se o saque falhar, nada muda nas duas.
Atenção: Pense no que acontece se o depósito falhar depois do saque. Esse problema tem nome: atomicidade (você vai ver de novo em SQL e em `@Transactional`).
-
- ☐ **85. Funcionário e aumento**
`Funcionario(nome, salario)` com `aplicarAumento(BigDecimal percentual)`. Percentual negativo é inválido.
Exemplo: 2000 com 10% → 2200.00.
-
- ☐ **86. Herança: Animal**
Classe `Animal(nome)` com `emitirSom()`. Cachorro e Gato herdam e sobrescrevem com `@Override`. Use `super(nome)` no construtor.
Atenção: Tire o `@Override` e erre o nome do método (`emitirSon`). Veja que compila e o comportamento quebra calado — é para isso que a anotação existe.
-
- ☐ **87. Polimorfismo**
Crie `List<Animal>` com cachorros e gatos e chame `emitirSom()` num loop, sem nenhum `if` ou `instanceof`.
Atenção: Explique num comentário: o método chamado é decidido em tempo de execução pelo tipo real do objeto.
-
- ☐ **88. Classe abstrata Funcionario**
`Funcionario` abstrato com `abstract BigDecimal calcularSalario()`. Subclasses: `Gerente` (base + bônus fixo), `Desenvolvedor` (base + horas extras × valor hora), `Estagiario` (bolsa fixa). `FolhaPagamento.total(List<Funcionario>)` soma tudo.
Exemplo: Teste cada subclasse e o total da folha.
Atenção: `new Funcionario()` não compila. Explique por quê.
-

- ☐ **89. Interface Autenticavel**
Interface Autenticavel com boolean autenticar(String senha). Gerente e Cliente implementam. Um SistemaInterno.login(Autenticavel a, String senha) aceita os dois.
Atenção: Não guarde a senha em texto puro: guarde o hash (MessageDigest com SHA-256) e compare hashes. No Spring você vai usar BCrypt.

- ☐ **90. Interface ou classe abstrata**
Adicione um método default na interface. Depois escreva no README, com suas palavras, quando usar interface e quando usar classe abstrata, com um exemplo do seu código.
Atenção: Pergunta de entrevista quase garantida.

- ☐ **91. Composição: ItemPedido**
ItemPedido(produto, quantidade, precoUnitario) com subtotal(). O precoUnitario é copiado do produto no momento da compra.
Atenção: Por que copiar o preço? Se o produto mudar de preço amanhã, o pedido de ontem não pode mudar de valor.

- ☐ **92. Pedido**
Pedido(id, cliente) com adicionarItem(produto, quantidade), removerItem(produtoId) e total(). Adicionar o mesmo produto duas vezes soma a quantidade em vez de criar outro item. Quantidade ≤ 0 é inválida.
Exemplo: 2x Camisa R\$ 50 + 1x Boné R\$ 30 → total R\$ 130.

- ☐ **93. Estoque**
Estoque com entrada(produtoId, qtd), saida(produtoId, qtd) e quantidade(produtoId). Saída maior que o disponível lança exceção e não altera nada.

- ☐ **94. Carrinho para Pedido**
Carrinho com adicionar/remover/limpar e finalizar(Estoque): valida estoque de todos os itens, dá baixa e gera um Pedido. Se um item não tiver estoque, nenhum sai.
Atenção: Mesma ideia de atomicidade da transferência.

- ☐ **95. Strategy: cupons de desconto**
Interface Desconto com BigDecimal aplicar(BigDecimal total). Implementações: SemDesconto, DescontoPercentual, DescontoValorFixo (nunca deixa o total negativo). O Pedido recebe um Desconto.
Atenção: Adicionar um novo tipo de cupom não pode exigir mexer no Pedido. Isso é polimorfismo trocando if/else — e o “O” do SOLID.

- ☐ **96. Relacionar Cliente, Pedido e Produto**
Cliente tem lista de pedidos. Implemente Cliente.totalGasto() e Cliente.pedidosAcimaDe(valor).
Atenção: Decida quem conhece quem (Cliente conhece Pedido? Pedido conhece Cliente? os dois?) e justifique.

- ☐ **97. Desafio: sistema de vendas no console**
Menu em loop: cadastrar produto, dar entrada em estoque, montar carrinho, aplicar cupom, finalizar pedido, listar pedidos. Use encapsulamento, composição, herança, polimorfismo e interfaces.
Atenção: Critério de pronto: nenhuma regra de negócio no main, todas as regras com teste, e o programa não fecha com exceção em nenhuma entrada.

98-113 | Exceptions

No Spring, erro de negócio vira resposta HTTP 400/404/409. Aqui você aprende a modelar e tratar do jeito certo.

❑ 98. Hierarquia de exceções

Desenhe (no README ou em comentário) a árvore Throwable → Error / Exception → RuntimeException. Dê 2 exemplos de cada e diga quais são checked.

Atenção: Checked = o compilador obriga a tratar ou declarar. Unchecked = não obriga.

❑ 99. Prever a saída

Escreva um método com try que dá return 1, catch que dá return 2 e finally que imprime algo. Antes de rodar, escreva o que você acha que imprime. Depois coloque um return 3 no finally e repita.

Atenção: return no finally engole exceções. Nunca faça isso — mas saiba explicar.

❑ 100. Divisão segura

dividir(int a, int b) que lança ArithmeticException com mensagem clara quando b == 0. No main, trate e mostre mensagem amigável.

❑ 101. Ler número até ser válido

Loop que pede idade até o usuário digitar um inteiro. Trate InputMismatchException.

Exemplo: "abc" → Digite um número válido e pergunta de novo.

Atenção: Sem input.nextLine() no catch, o Scanner entra em loop infinito com o mesmo token inválido. Descubra por quê.

❑ 102. Converter String para int

int converter(String s) que trata NumberFormatException. Depois faça a versão Optional<Integer> tentarConverter(String s).

Exemplo: "42" → 42; "4a2" → Optional.empty(); null → Optional.empty().

❑ 103. Múltiplos catch e multi-catch

Um método que pode lançar NumberFormatException e ArrayIndexOutOfBoundsException (lê args[0] e converte). Trate com dois catch e depois com um catch (A | B e).

Atenção: A ordem importa: catch (Exception e) antes de um mais específico não compila.

❑ 104. finally

Simule abrir e fechar um recurso: imprima "abrindo", faça uma operação que pode falhar e garanta "fechando" com finally, com e sem erro.

❑ 105. Exceções de negócio

Crie NegocioException extends RuntimeException e as filhas SaldoInsuficienteException, EstoqueInsuficienteException e RecursoNaoEncontradoException. Troque os IllegalStateException do módulo de POO por elas.

Atenção: A mensagem deve ser útil: Saldo insuficiente: saldo 70.00, saque 100.00 — não Erro.

❑ 106. throw vs throws

validarIdade(int idade) que usa throw. lerArquivo(Path p) throws IOException que declara e propaga. Explique a diferença num comentário.

❑ 107. Checked exception própria

Crie ConfiguracaoException extends Exception. Um método de leitura de configuração declara throws ConfiguracaoException e quem chama decide tratar.

Atenção: Em código moderno (e no Spring) se prefere unchecked. Saiba defender os dois lados.

☐ **108. Exception chaining**

Ao converter uma configuração “porta=abc”, capture o `NumberFormatException` e lance `ConfiguracaoException("Porta inválida: abc", e)`. Imprima o stack trace e ache o `Caused by`.

Atenção: Nunca perca a causa original: `throw new X(msg)` sem o `e` e apaga a pista do bug.

☐ **109. try-with-resources e AutoCloseable**

Crie uma classe `Conexao` implements `AutoCloseable` que imprime ao abrir e fechar. Use duas no mesmo `try-with-resources` e veja a ordem de fechamento. Faça o `close()` lançar exceção e ache as `getSuppressed()`.

☐ **110. Antipadrões**

Reescreva este código ruim: `try { ... } catch (Exception e) { }, catch (Exception e) { e.printStackTrace(); return null; }` e um loop que usa exceção para sair. Explique o que cada um esconde.

☐ **111. Tratamento centralizado**

No sistema de vendas, faça cada comando do menu lançar `NegocioException` quando a regra falha. Um único `try/catch` no loop do menu mostra a mensagem e continua rodando.

Atenção: É exatamente o que o `@RestControllerAdvice` faz no Spring: um lugar só converte exceção em resposta.

☐ **112. Desafio: banco com validações**

Contas, depósito, saque, transferência e extrato, com exceções de negócio e testes com `assertThrows` para cada regra. O programa nunca fecha com exceção.

☐ **113. Desafio: pedidos sem exceção escapando**

Pegue o sistema de vendas e garanta: produto inexistente → `RecursoNaoEncontradoException`; estoque insuficiente → `EstoqueInsuficienteException`; entrada inválida → mensagem e nova tentativa. Escreva um teste para cada caminho de erro.

114-138 | Collections

Saber QUAL coleção usar e o custo de cada operação é o que a entrevista testa.

☐ **114. List: CRUD de pessoas**

`List<Pessoa>` com adicionar, listar, buscar por nome (ignorando maiúsculas), atualizar idade e remover. Mostre a quantidade depois de cada operação.

Atenção: Em `List<Integer>`, `lista.remove(1)` remove o ÍNDICE 1, não o número 1. Use `remove(Integer.valueOf(1))` e prove com um teste.

☐ **115. Formas de percorrer**

Percorra a mesma lista com `for` de índice, `for-each`, `Iterator` e `forEach` com `lambda`. Diga quando cada uma faz sentido.

☐ **116. Mais velha e mais nova**

`Pessoa maisVelha(List<Pessoa>)` e `maisNova` com loop. Lista vazia lança exceção. Empate retorna a primeira.

- ☐ **117. Comparator**
Ordene pessoas por nome, por idade, por idade decrescente e por idade e depois nome.
Exemplo:
`lista.sort(Comparator.comparing(Pessoa::getIdade).thenComparing(Pessoa::getNome));`
Atenção: Faça primeiro com classe anônima, depois com lambda, depois com `Comparator.comparing`. Veja a evolução.

- ☐ **118. Comparable**
`Pessoa` implements `Comparable<Pessoa>` com ordem natural por nome. Use `Collections.sort(lista)` e um `TreeSet<Pessoa>`.
Atenção: Se o `compareTo` diz que duas pessoas diferentes com o mesmo nome são “iguais”, o `TreeSet` descarta uma. Prove e explique.

- ☐ **119. equals e hashCode**
Coloque duas `Pessoa` com mesmo CPF num `HashSet` sem `equals/hashCode` — entram as duas. Implemente pelos campos que identificam (CPF) e repita. Teste o contrato: iguais → mesmo `hashCode`.
Atenção: Mude o CPF de uma pessoa DEPOIS de colocar no `HashSet` e tente achá-la com `contains`. Explique por que sumiu.

- ☐ **120. Os três Sets**
Adicione os mesmos nomes (com repetidos, fora de ordem) em `HashSet`, `LinkedHashSet` e `TreeSet`. Imprima e compare a ordem.

- ☐ **121. Remover duplicados com Set**
Refaça o “remover duplicados do array” em uma linha preservando a ordem.
Exemplo: `new ArrayList<>(new LinkedHashSet<>(lista))`

- ☐ **122. HashMap: agenda**
`Agenda Map<String, String>` (nome → telefone): adicionar, buscar, atualizar, remover e listar em ordem alfabética.
Atenção: Use `getOrDefault`, `putIfAbsent` e `containsKey`. Para listar ordenado, use `TreeMap`.

- ☐ **123. Percorrer Map**
Percorra com `keySet()`, `values()` e `entrySet()`. Diga por que `entrySet()` é o jeito certo quando você precisa da chave e do valor.

- ☐ **124. Frequência de palavras**
`Map<String, Integer> frequencia(String texto)` ignorando caixa e pontuação, e depois `top3`.
Exemplo: Use `map.merge(palavra, 1, Integer::sum)`.

- ☐ **125. Frequência de caracteres**
Conte cada letra de um texto num `TreeMap<Character, Integer>` e imprima um histograma com `*`.
Exemplo: a: *****

- ☐ **126. Agrupar funcionários por departamento**
`Map<String, List<Funcionario>>` usando `computeIfAbsent`. Depois calcule a folha por departamento.

- ☐ **127. Dois que somam em O(n)**
Refaça o “dois números que somam o alvo” com `HashMap` numa única passada. Compare o tempo com a versão $O(n^2)$ usando 50.000 elementos.
Atenção: Troca clássica: memória por velocidade.

-
- ☐ **128. Repositório em memória**
ProdutoRepository com Map<Long, Produto>: save (gera id se for novo), findById (retorna Optional<Produto>), findAll, deleteById, existsById.
Atenção: Esses são exatamente os nomes do JpaRepository do Spring. Você está escrevendo o que o Spring vai te dar pronto.
-
- ☐ **129. Pedidos: consultas**
Com List<Pedido>: total de cada pedido, pedido de maior valor, pedidos de um cliente, ordenar por valor e por data, agrupar por cliente. Tudo com loop (streams vêm depois).
-
- ☐ **130. Paginação**
Pagina<T> paginar(List<T> itens, int pagina, int tamanho) retornando record
Pagina<T>(List<T> conteudo, int pagina, int tamanho, long totalElementos, int totalPaginas).
Exemplo: 23 itens, página 2, tamanho 10 → 3 itens, totalPaginas 3.
Atenção: Página fora do intervalo retorna conteúdo vazio, não exceção. Esse é o formato do Page do Spring.
-
- ☐ **131. ConcurrentModificationException**
Remova pessoas menores de idade dentro de um for-each e veja a exceção. Corrija de três formas: Iterator.remove(), removeIf e criando uma lista nova.
-
- ☐ **132. Listas imutáveis**
Tente add num List.of(...), num Collections.unmodifiableList(...) e num List.copyOf(...). Mude a lista original e veja qual delas “muda junto”.
-
- ☐ **133. Fila e pilha**
Fila de atendimento com ArrayDeque (offer/poll). Pilha para validar parênteses balanceados (push/pop).
Exemplo: "{[()]}" → true; "([)]" → false; "({" → false.
Atenção: Não use a classe Stack antiga. Esse exercício cai em teste técnico.
-
- ☐ **134. Big-O das coleções**
Meça contains em ArrayList, LinkedList e HashSet com 100.000 elementos. Monte uma tabela no README com o custo de get, add, remove e contains de cada uma.
-
- ☐ **135. Como o HashMap funciona**
Explique no README: hash, buckets, colisão, por que hashCode ruim deixa o map lento e o que acontece com uma chave mutável.
Atenção: Pergunta de entrevista de júnior para pleno. Saber isso te destaca.
-
- ☐ **136. Escolher a coleção certa**
Para cada cenário escolha e justifique: histórico de pedidos em ordem; CPFs únicos; busca de produto por código; ranking ordenado por pontos; fila de processamento; cache de CEP.
-
- ☐ **137. Desafio: pedidos com Collections**
Sistema de pedidos usando repositórios em memória (Map), Set para evitar clientes duplicados por CPF, List de itens, exceções de negócio e paginação na listagem. Tudo testado.
-
- ☐ **138. Desafio: estoque**
Estoque por produto com histórico de movimentações (entrada/saída, data, quantidade), relatório de produtos abaixo do mínimo e bloqueio de saída sem saldo.
-

139-153 | Generics, Enum e Date/Time

☐ 139. Classe genérica

Caixa<T> com guardar, abrir e estaVazia. Depois Par<A, B> com primeiro e segundo.

Atenção: Mostre o erro de compilação que você ganha e o `ClassCastException` que você evita, comparando com uma versão que usa `Object`.

☐ 140. Método genérico com limite

`static <T extends Comparable<T>> T maior(List<T> itens)` funcionando para `Integer`, `String` e `Pessoa`.

☐ 141. Wildcards

`double somar(List<? extends Number> numeros)` aceitando `List<Integer>` e `List<Double>`. Tente adicionar um elemento dentro do método e entenda o erro.

Atenção: Regra PECS: *extends* para ler, *super* para escrever.

☐ 142. Repositório genérico

`interface Repositorio<T, ID>` e `RepositorioMemoria<T extends Entidade<ID>, ID>`. Use para `Produto` e `Cliente` sem duplicar código.

Atenção: É a assinatura do `JpaRepository<T, ID>`.

☐ 143. Enum StatusPedido com transições

`CRIADO`, `PAGO`, `ENVIADO`, `ENTREGUE`, `CANCELADO` com `boolean podeIrPara(StatusPedido novo)`. Só pode cancelar se `CRIADO` ou `PAGO`. Pedido usa isso em `alterarStatus`.

Exemplo: `ENVIADO` → `CANCELADO` lança `NegocioException`.

Atenção: Teste todas as transições com `@ParameterizedTest` + `@EnumSource/@CsvSource`.

☐ 144. Enum com código e descrição

`FormaPagamento(int codigo, String descricao)` com `static FormaPagamento porCodigo(int codigo)` que lança exceção para código inexistente.

Atenção: `FormaPagamento.valueOf("pix")` lança `IllegalArgumentException` (é case-sensitive). Teste.

☐ 145. Enum com comportamento

Cada `FormaPagamento` calcula sua taxa: `PIX 0%`, `CARTAO 2,5%`, `BOLETO R$ 3,00 fixo`. Sem `switch` fora do enum.

☐ 146. EnumMap

Conte pedidos por status com `EnumMap<StatusPedido, Integer>`.

☐ 147. Idade exata com LocalDate

`int idade(LocalDate nascimento, LocalDate hoje)` com `Period`. Teste aniversário hoje, amanhã e 29/02.

Atenção: Receba hoje como parâmetro (ou um `Clock`). Código que chama `LocalDate.now()` lá dentro não é testável.

☐ 148. LocalDateTime e Duration

Calcule quanto tempo um pedido levou de `CRIADO` a `ENTREGUE` em dias e horas.

☐ 149. Formatar e converter

Converta `"26/09/2026"` para `LocalDate` e de volta com `DateTimeFormatter.ofPattern("dd/MM/yyyy")`. Trate `DateTimeParseException` para `"31/02/2026"`.

- ☐ **150. Comparar e filtrar por período**
List<Pedido> entre(LocalDate inicio, LocalDate fim) com as duas pontas inclusivas. Use isBefore, isAfter, isEqual.
Atenção: Com LocalDateTime, “até dia 30” precisa pegar 30/09 às 23:59. Use fim.plusDays(1).atStartOfDay() exclusivo.

- ☐ **151. Prazo em dias úteis**
LocalDate prazoEntrega(LocalDate compra, int diasUteis, Set<LocalDate> feriados) pulando sábado, domingo e feriados.
Exemplo: Compra sexta, 2 dias úteis → terça.

- ☐ **152. Fuso horário**
Mostre o mesmo Instant em UTC e em America/Recife. Explique no README por que o banco deve guardar em UTC.

- ☐ **153. Desafio: relatório de pedidos**
Relatório por período e status: quantidade, faturamento, ticket médio e prazo médio de entrega, usando Generics, enum e Date/Time.

154-169 | Lambda, Streams e Optional

Refaça consultas que você já fez com loop. Se a versão com stream ficar ilegível, o loop ganha.

- ☐ **154. Interfaces funcionais**
Use Predicate, Function, Consumer, Supplier e BiFunction num exemplo cada com produtos. Depois crie sua própria @FunctionalInterface.

- ☐ **155. Method reference**
Troque lambdas por method references dos 4 tipos: estático, de instância de um objeto, de instância de um tipo e construtor.
Exemplo: Produto::getNome, System.out::println, ArrayList::new

- ☐ **156. filter, map e toList**
Nomes dos produtos acima de R\$ 100 em maiúsculas, ordenados.

- ☐ **157. Números**
Com mapToInt/mapToDouble: soma, média, máximo e mínimo de idades. Use IntSummaryStatistics para pegar tudo de uma vez.
Atenção: average() retorna OptionalDouble — lista vazia não tem média.

- ☐ **158. Somar BigDecimal**
Total de um pedido com stream sobre itens.
Exemplo: itens.stream().map(ItemPedido::subtotal).reduce(BigDecimal.ZERO, BigDecimal::add)

- ☐ **159. sorted, distinct, limit, skip**
Top 5 produtos mais caros sem repetir categoria; página 2 de uma lista usando skip/limit.

- ☐ **160. anyMatch, allMatch, noneMatch**
Existe produto sem estoque? Todos os pedidos estão pagos? Nenhum cliente é menor de idade?
Atenção: allMatch numa lista vazia retorna true. Saiba explicar.

- ☐ **161. findFirst e Optional**
Primeiro pedido cancelado de um cliente, retornando Optional<Pedido>.

- ☐ **162. groupingBy**
Produtos por categoria; quantidade por categoria (counting); faturamento por cliente (reducing ou mapping + soma); nomes por categoria (mapping).

- ☐ **163. partitioningBy**
Separe pedidos acima e abaixo de R\$ 200 num `Map<Boolean, List<Pedido>>`.

- ☐ **164. toMap e chave duplicada**
`Map<String, Produto>` por nome. Crie dois produtos com o mesmo nome e veja o `IllegalStateException`. Resolva com a função de merge.

- ☐ **165. flatMap**
A partir de uma lista de pedidos, a lista de produtos distintos vendidos e o produto mais vendido em quantidade.

- ☐ **166. Stream é preguiçoso e de uso único**
Use peek para mostrar que nada roda sem operação terminal. Tente usar o mesmo stream duas vezes e veja a exceção.

- ☐ **167. Optional do jeito certo**
buscarPorId retorna Optional. Use map, filter, orElse, orElseGet, orElseThrow e ifPresentOrElse.
Atenção: `orElse(criarPadrao())` executa `criarPadrao()` SEMPRE; `orElseGet` só se vazio. Prove com um print. Não use Optional como atributo nem parâmetro, e evite `get()`.

- ☐ **168. Refatorar para stream (e de volta)**
Reescreva 5 consultas do módulo de Collections com streams. Escolha uma que ficou pior e volte para loop, justificando.

- ☐ **169. Desafio: sistema de funcionários**
Cadastro de funcionários com relatórios via streams: folha por departamento, top 3 salários, média por cargo, busca por nome com Optional e aumento em lote. Tudo testado.

170-189 | Java Core e design antes do Spring

Imutabilidade, records, memória, arquivos, logs e como organizar um projeto em camadas.

- ☐ **170. Classe imutável**
Endereco imutável: classe final, campos private final, sem setters, cópia defensiva de listas, métodos com `Rua(...)` que retornam um novo objeto.
Atenção: Explique por que objeto imutável é seguro para compartilhar entre threads.

- ☐ **171. final em tudo**
Mostre final em variável local, atributo, parâmetro, método (não pode sobrescrever) e classe (não pode herdar). Tente violar cada um e registre o erro de compilação.
Atenção: `final List<String> x` ainda permite `x.add(...)`. Final protege a referência, não o conteúdo.

- ☐ **172. Record**
Transforme Endereco em record com construtor compacto validando CEP (8 dígitos). Veja o que o record gera sozinho (`equals`, `hashCode`, `toString`, acessores).

- ☐ **173. Class ou record**
Para Produto (tem estoque que muda), Endereco, ItemPedido e um DTO ProdutoResponse, decida class ou record e justifique.
Atenção: No Spring, DTO quase sempre é record; entidade JPA nunca é.

- ☐ **174. Java passa por valor**
Crie um método que recebe uma Pessoa e (1) muda a idade, (2) faz `p = new Pessoa(...)`. Veja qual das duas mudanças o chamador enxerga. Faça o mesmo com `int`.
Atenção: Java é SEMPRE passagem por valor — o valor de uma variável de objeto é a referência. Pergunta clássica.

- ☐ **175. Escopo e sombreamento**
Mostre variável de bloco, de método, atributo e um parâmetro com o mesmo nome do atributo (por isso `this.nome = nome`). Mostre variável usada em lambda que precisa ser efetivamente final.

- ☐ **176. Stack, heap e garbage collector**
Provoque `StackOverflowError` com recursão sem caso base. Explique no README: o que vai na stack, o que vai no heap e quando um objeto pode ser coletado.

- ☐ **177. Recursão**
Fatorial e Fibonacci recursivos. Meça `fib(40)` ingênuo e depois com memoização num `HashMap`.

- ☐ **178. Classe utilitária**
Validacoes com métodos static (`isCpfValido`, `isEmailValido`, `exigirPositivo`) e construtor private para ninguém instanciar.
Atenção: Static é bom para função pura. Ruim quando esconde estado ou dependência (dificulta teste).

- ☐ **179. Ler e escrever arquivo**
Com `java.nio.file.Files`: escreva uma lista de produtos num `produtos.txt` e leia de volta. Trate `IOException`.

- ☐ **180. Importar CSV com erros**
Leia `produtos.csv` (`id;nome;preco;estoque`). Linhas inválidas não param a importação: gere um relatório “linha 7: preço inválido 'abc'” e importe o resto.
Exemplo: Resultado: `ResultadoImportacao(importados=48, erros=[...])`.
Atenção: Arquivo com cabeçalho, linha vazia no fim e acentos (UTF-8). Teste cada caso.

- ☐ **181. Exportar relatório**
Gere `relatorio-vendas.csv` com faturamento por dia usando `StringBuilder` e `Files.writeString`.

- ☐ **182. Arquivo de configuração**
Leia `app.properties` (`url do banco`, `usuário`, `porta`). Chave obrigatória ausente lança `ConfiguracaoException`; porta inválida também. Valores podem ser sobrescritos por variável de ambiente.
Atenção: Exatamente o papel do `application.properties` do Spring.

- ☐ **183. Logging com SLF4J**
Adicione SLF4J + Logback pelo Maven. Troque os `System.out` de erro por `log.info`, `log.warn`, `log.error`. Use parâmetros: `log.info("Pedido {} criado", id)`.
Atenção: Nunca logue senha, token ou CPF completo. Configure o nível para esconder DEBUG.

☐ **184. Refatorar código duplicado**

Pegue o seu sistema de vendas e extraia métodos e classes onde houver repetição. Nomes devem dizer o que fazem sem comentário.

Atenção: Métodos com mais de ~20 linhas ou 3 níveis de if são candidatos.

☐ **185. Composição em vez de herança**

Crie Notificador (interface) com EmailNotificador e ConsoleNotificador. PedidoService RECEBE um notificador no construtor em vez de herdar de algo.

Atenção: Esse é o “D” do SOLID (depende de abstração).

☐ **186. Builder**

Pedido.builder().cliente(c).item(p, 2).desconto(d).build() validando no build().

Atenção: Você vai ver isso no Lombok (@Builder). Entenda antes de usar pronto.

☐ **187. Arquitetura em camadas sem Spring**

Reorganize o sistema em packages: model, repository, service, dto e app (console). Regra: app só fala com service; service fala com repository; model não depende de ninguém.

Atenção: Controller → Service → Repository é a estrutura de 90% das APIs Spring.

☐ **188. DTO vs entidade**

Crie CriarPedidoRequest e PedidoResponse como records. O service recebe o request, converte para entidade, e devolve o response. A entidade nunca sai do service.

☐ **189. Injeção de dependência na mão**

Crie uma classe Aplicacao que monta tudo: repositórios → services → console, passando dependências por construtor. Nenhuma classe dá new nas suas dependências.

Atenção: É isso que o Spring automatiza com @Service e @Autowired. Se você entender aqui, o Spring deixa de ser mágica.

190-195 | Testes com Mockito **NOVO**

É assim que se testa um Service no Spring.

☐ **190. Primeiro mock**

Adicione mockito-junit-jupiter. Teste PedidoService com @ExtendWith(MockitoExtension.class), @Mock PedidoRepository e @InjectMocks PedidoService.

☐ **191. when / thenReturn**

Simule findById(1L) retornando um pedido e teste buscarPorId.

☐ **192. Caminho de erro**

Simule Optional.empty() e teste que o service lança RecursoNaoEncontradoException com a mensagem certa.

☐ **193. verify**

Ao criar pedido, verifique que save foi chamado uma vez e que o notificador foi chamado. Quando a validação falha, verifique never().

☐ **194. ArgumentCaptor**

Capture o pedido passado para save e confira status CRIADO, total e data.

☐ **195. Mock vs stub vs fake**

Troque o mock do repositório por um fake (o repositório em memória) num teste. Escreva no README quando usar cada um e a diferença entre teste unitário e de integração.

196-200 | Concorrência básica **NOVO**

Não precisa dominar, mas as perguntas abaixo caem em entrevista.

☐ **196. Thread e Runnable**

Crie duas threads que imprimem de 1 a 5 com nomes diferentes. Rode várias vezes e observe a ordem mudar.

☐ **197. Race condition**

Duas threads incrementam o mesmo contador 100.000 vezes cada. O resultado não é 200.000. Explique por quê (count++ não é atômico).

☐ **198. synchronized e AtomicInteger**

Corrija o contador de duas formas: synchronized e AtomicInteger. Compare o tempo.

☐ **199. ExecutorService**

Use um pool de 4 threads para “processar” 20 pedidos (com Thread.sleep) e compare com a versão sequencial. Colete resultados com Future.

☐ **200. Coleções thread-safe**

Faça duas threads escreverem num HashMap e num ConcurrentHashMap. Explique por que o servidor web precisa se preocupar com isso (cada requisição pode ser uma thread).

201-203 | Docker mínimo **NOVO**

Só o suficiente para subir o PostgreSQL e, depois, sua aplicação.

☐ **201. Primeiro container**

Instale o Docker. Rode `docker run hello-world`. Aprenda `docker ps`, `docker ps -a`, `docker logs`, `docker stop`, `docker rm`.

☐ **202. PostgreSQL no Docker**

Suba um PostgreSQL com usuário, senha, banco e porta 5432, com volume para não perder os dados. Entre com `docker exec -it <container> psql -U <usuario> loja`.

Atenção: Apague o container e suba de novo: sem volume, os dados somem.

☐ **203. docker compose**

Escreva um `docker-compose.yml` com o PostgreSQL. `docker compose up -d` e `down`. Coloque no repositório com instrução no README.

204-214 | JDBC: Java falando com o banco **NOVO**

Faça depois de Exceptions e de S1-S31. É o que o Spring Data esconde de você.

☐ **204. Conectar**

Adicione o driver `org.postgresql:postgresql` no pom. Abra uma Connection com DriverManager e imprima a versão do banco.

Atenção: URL, usuário e senha vêm do `app.properties` ou de variável de ambiente — nunca no código nem no Git.

☐ **205. Listar**

`List<Cliente> findAll()` com PreparedStatement e ResultSet, mapeando cada linha para objeto.

☐ **206. SQL Injection**

Faça uma busca por nome concatenando String e “invada” com ' OR '1'='1. Refaça com PreparedStatement e mostre que o ataque não funciona.

Atenção: Pergunta de segurança obrigatória em entrevista.

☐ **207. Inserir e pegar o id**

Cliente save(Cliente c) com INSERT ... RETURNING id (ou RETURN_GENERATED_KEYS) devolvendo o cliente com id.

☐ **208. Atualizar e apagar**

update e deleteById retornando quantas linhas foram afetadas. Zero linhas → RecursoNaoEncontradoException.

☐ **209. Fechar recursos**

Garanta que Connection, PreparedStatement e ResultSet fecham sempre com try-with-resources. Provoque um erro no meio e confirme.

☐ **210. Mapear relacionamento**

Optional<Pedido> findById(long id) trazendo o pedido e seus itens com JOIN, montando o objeto completo.

Atenção: Evite fazer uma query para o pedido e depois uma para CADA item (problema N+1 — você vai ver de novo no JPA).

☐ **211. Transação na mão**

Criar pedido = inserir pedido + itens + baixar estoque. Use setAutoCommit(false), commit() e rollback() no catch. Force erro no último item e confirme que nada foi gravado.

Atenção: É o que o @Transactional faz por baixo.

☐ **212. Repository com interface**

ClienteRepository (interface) com ClienteRepositoryJdbc e ClienteRepositoryMemoria. O service não sabe qual está usando.

☐ **213. Traduzir SQLException**

Converta SQLException numa DadosException unchecked com causa encadeada. Violação de UNIQUE (email duplicado) vira NegocioException("E-mail já cadastrado").

Atenção: O SQLState 23505 indica unique violation no PostgreSQL.

☐ **214. Teste de integração**

Um teste que sobe o banco de teste (o do Docker), limpa as tabelas no @BeforeEach e testa o repositório de verdade.

Atenção: Pesquise Testcontainers: é o que se usa em projeto Spring sério.

215-224 | HTTP, REST e JSON **NOVO**

É o que você vai construir no Spring. Aqui você faz sem framework.

☐ **215. Anatomia do HTTP**

Com curl -v numa API pública, identifique: método, URL, headers da requisição, status, headers da resposta e corpo.

Exemplo: curl -v https://viacep.com.br/ws/53401000/json/

☐ **216. Verbos e idempotência**

Monte uma tabela: GET, POST, PUT, PATCH, DELETE — para que serve, se tem corpo, se é idempotente e se é seguro.

☐ **217. Status codes**

Diga o status certo para: listou; criou; apagou sem retorno; JSON mal formado; campo inválido; sem login; logado sem permissão; não achou; e-mail já existe; erro inesperado.

Atenção: 200, 201, 204, 400, 422/400, 401, 403, 404, 409, 500. Saber isso de cor é obrigatório.

☐ **218. Desenhar uma API REST**

No papel, desenhe os endpoints da loja: CRUD de produtos, busca por categoria, paginação, pedidos de um cliente e mudança de status do pedido.

Exemplo: GET /produtos?page=0&size=10, GET /produtos/{id}, POST /produtos, PATCH /pedidos/{id}/status, GET /clientes/{id}/pedidos

Atenção: Substantivo no plural, sem verbo na URL (/criarProduto está errado).

☐ **219. Postman ou Insomnia**

Faça as mesmas chamadas do curl numa ferramenta visual, com uma coleção salva e uma variável baseUrl.

☐ **220. JSON na mão**

Escreva à mão o JSON de um pedido com cliente, itens, total e data. Valide num validador online. Diga os 6 tipos do JSON.

☐ **221. Jackson**

Adicione Jackson. Converta Pedido para JSON e de volta com ObjectMapper. Registre o JavaTimeModule para LocalDate funcionar e veja como sai o BigDecimal.

☐ **222. Cliente HTTP em Java**

Com java.net.http.HttpClient, busque um CEP no ViaCEP e converta para record Endereco. Trate CEP inexistente ({ "erro": true }), status diferente de 200 e timeout.

☐ **223. Mini API sem Spring**

Com com.sun.net.httpserver.HttpServer (vem no JDK), exponha GET /produtos, GET /produtos/{id} (404 se não existir) e POST /produtos (201 com header Location). Use seu ProdutoService e Jackson.

Atenção: Faça funcionar com curl. Depois disso, o @RestController vai parecer um atalho — e é.

☐ **224. Erros padronizados**

Na mini API, toda exceção vira JSON { "status": 404, "erro": "Produto 9 não encontrado", "timestamp": "... "}. Validação vira 400 com a lista de campos inválidos.

Atenção: Um lugar só faz essa tradução — o mesmo papel do @RestControllerAdvice.

225 | Projeto final

O portfólio que prova que você está pronto para o Spring.

☐ **225. Sistema de pedidos completo sem Spring**

API HTTP (HttpServer) + PostgreSQL via JDBC + Docker Compose. Recursos: clientes, produtos, estoque, pedidos com status (enum com transições), cupons (Strategy) e relatório por período (streams). Camadas model / repository / service / dto / http. Exceções de negócio viram JSON com status certo. Logs com SLF4J. Testes: unitários com JUnit + Mockito nos services, integração nos repositórios. README com diagrama das tabelas, como subir e exemplos de curl.

Atenção: Critério de pronto: git clone + docker compose up + mvn test funcionando numa máquina que não é a sua.

TRILHAS PARALELAS

S1-S38 | SQL com PostgreSQL **NOVO**

Trilha PARALELA: começa amanhã, 40 min por dia, sem pular dia. Tudo no mesmo domínio da loja. Escreva cada query em arquivo .sql versionado no Git. Sem ORM, sem IA gerando query.

☐ S1. Ambiente

Suba o PostgreSQL (Docker ou instalado). Acesse pelo psql e por um cliente gráfico (DBaver ou pgAdmin). No psql, use \l, \c, \dt, \d tabela, \q.

☐ S2. Banco e schema

Crie o banco loja. Explique a diferença entre database, schema e tabela, e para que serve o schema public.

☐ S3. Tabela cliente

cliente(id, nome, email, cpf, data_nascimento, criado_em) com tipos corretos: BIGINT GENERATED ALWAYS AS IDENTITY, VARCHAR(n), CHAR(11), DATE, TIMESTAMP com DEFAULT now().

Atenção: Justifique cada tipo. VARCHAR(255) para tudo é preguiça.

☐ S4. Tabela produto

produto(id, nome, categoria, preco, estoque, ativo) com preco NUMERIC(10,2) e ativo BOOLEAN DEFAULT true.

Atenção: FLOAT para dinheiro é erro eliminatório. Explique por quê com o que você viu no módulo de BigDecimal.

☐ S5. Constraints

Adicione PRIMARY KEY, NOT NULL, UNIQUE (email), UNIQUE (cpf), CHECK (preco > 0) e CHECK (estoque >= 0). Tente inserir dados que violam cada uma e leia o erro.

Atenção: A regra no banco é a última linha de defesa: mesmo com bug no Java, dado inválido não entra.

☐ S6. Tabela pedido

pedido(id, cliente_id, status, criado_em) com FOREIGN KEY para cliente e CHECK (status IN ('CRIADO', 'PAGO', 'ENVIADO', 'ENTREGUE', 'CANCELADO')).

☐ S7. Tabela item_pedido

item_pedido(pedido_id, produto_id, quantidade, preco_unitario) com chave primária composta (pedido_id, produto_id) e duas FKs.

Atenção: Explique por que preco_unitario fica no item e não só no produto (mesmo motivo do módulo de POO).

☐ S8. ALTER TABLE

Adicione telefone em cliente, renomeie para celular, mude o tamanho, e depois remova. Adicione uma constraint numa tabela que já tem dados que a violam e veja o erro.

☐ S9. Popular o banco

Script data.sql com 15 clientes, 25 produtos em 5 categorias, 40 pedidos em meses diferentes e 100 itens. Inclua clientes sem pedido e produtos nunca vendidos.

Atenção: Sem dados variados, suas consultas “funcionam” por acaso.

☐ S10. SELECT e WHERE

Clientes nascidos antes de 2000; produtos entre R\$ 50 e R\$ 200 (BETWEEN); produtos das categorias A ou B (IN); clientes com “silva” no nome ignorando maiúsculas (ILIKE).

- ☐ **S11. Ordenar e paginar**
Produtos por preço decrescente e nome crescente. Página 3 com 5 itens por página (LIMIT/OFFSET).
Atenção: Escreva a fórmula do OFFSET a partir de page e size. Paginação sem ORDER BY retorna ordem imprevisível.

- ☐ **S12. UPDATE**
Aumente 10% o preço de uma categoria. Depois, num banco de teste, rode um UPDATE sem WHERE e veja o estrago.
Atenção: Hábito: escreva o SELECT com o WHERE primeiro, confira as linhas, depois troque por UPDATE.

- ☐ **S13. DELETE e FK**
Apague um produto nunca vendido. Tente apagar um cliente com pedidos e leia o erro. Compare ON DELETE RESTRICT, CASCADE e SET NULL, e diga qual usaria em pedido → cliente.
Atenção: Em sistema real quase nunca se apaga: use soft delete (ativo = false).

- ☐ **S14. NULL**
Clientes sem telefone. Mostre “não informado” no lugar de NULL com COALESCE. Prove que NULL = NULL não é verdadeiro.

- ☐ **S15. Agregações**
Quantidade de clientes, soma do estoque, preço médio, produto mais caro e mais barato.
Atenção: COUNT(*) vs COUNT(coluna) com NULL. Saiba a diferença.

- ☐ **S16. GROUP BY**
Quantidade de produtos e preço médio por categoria; quantidade de pedidos por status.
Atenção: Toda coluna do SELECT que não está numa agregação tem que estar no GROUP BY. Entenda o erro que aparece se não estiver.

- ☐ **S17. HAVING**
Categorias com preço médio acima de R\$ 100; clientes com mais de 3 pedidos.
Atenção: WHERE filtra linhas antes de agrupar, HAVING filtra grupos depois. Pergunta clássica.

- ☐ **S18. INNER JOIN**
Todos os pedidos com nome e e-mail do cliente.

- ☐ **S19. LEFT JOIN**
Clientes que nunca compraram; produtos que nunca foram vendidos.
Atenção: Explique a diferença de INNER, LEFT, RIGHT e FULL com um desenho. Pergunta clássica.

- ☐ **S20. JOIN de 3 tabelas**
Cada item vendido com número do pedido, nome do cliente, nome do produto, quantidade e subtotal.

- ☐ **S21. Total de cada pedido no SQL**
Total por pedido com SUM(quantidade * preco_unitario) e ordenado do maior para o menor.

- ☐ **S22. Produto mais vendido**
Top 5 produtos por quantidade vendida e top 5 por faturamento. Compare os dois rankings.

- ☐ **S23. Faturamento por mês**
Faturamento por mês de 2026 com `DATE_TRUNC('month', criado_em)`, só pedidos não cancelados.

- ☐ **S24. Subquery**
Produtos com preço acima da média geral; clientes cujo gasto total é maior que a média de gasto dos clientes.

- ☐ **S25. EXISTS vs IN**
Clientes que têm pelo menos um pedido PAGO, com `EXISTS` e com `IN`. Veja o que acontece com `NOT IN` quando a subquery retorna `NULL`.

- ☐ **S26. DISTINCT e UNION**
Categorias distintas vendidas; `UNION` vs `UNION ALL` com duas listas de e-mails.

- ☐ **S27. CASE WHEN**
Classifique pedidos em PEQUENO (< 100), MÉDIO (< 500) e GRANDE, e conte quantos de cada.

- ☐ **S28. CTE**
Reescreva a consulta de “clientes que gastam mais que a média” com `WITH` em etapas nomeadas.

- ☐ **S29. Window functions**
Ranking de clientes por gasto com `RANK()` e `ROW_NUMBER()`; o pedido mais caro de cada cliente com `PARTITION BY`; total acumulado por mês.
Atenção: É o que separa quem “sabe `SELECT`” de quem sabe `SQL`.

- ☐ **S30. Transação**
No `psql`: `BEGIN`, crie um pedido, insira itens, baixe estoque, `ROLLBACK` e confira que nada mudou. Repita com `COMMIT`.

- ☐ **S31. ACID e isolamento**
Explique Atomicidade, Consistência, Isolamento e Durabilidade com o exemplo do exercício anterior. Abra dois `psql` ao mesmo tempo e mostre que uma transação não vê dados não commitados da outra.

- ☐ **S32. Índices**
Gere 500.000 pedidos com `generate_series`. Rode `EXPLAIN ANALYZE` numa busca por `cliente_id`, crie um índice, rode de novo e compare.
Atenção: Explique por que índice acelera leitura e deixa escrita mais lenta, e por que a FK não cria índice sozinha no PostgreSQL.

- ☐ **S33. Normalização**
Pegue uma “planilhona” (pedido com nome do cliente, e-mail, produto1, produto2, produto3 na mesma linha) e normalize até a 3ª forma normal explicando cada passo.

- ☐ **S34. Modelagem**
Desenhe o DER da loja com cardinalidades 1:1, 1:N e N:N. Depois modele sozinho, do zero, um sistema de biblioteca (livro, autor, exemplar, usuário, empréstimo, multa).

- ☐ **S35. VIEW**
Crie `vw_vendas_mensais` e consulte como se fosse tabela. Explique quando uma view ajuda.

☐ **S36. Scripts reproduzíveis**

schema.sql e data.sql que recriam o banco do zero com um comando. Coloque no repositório.

Atenção: Isso é o conceito de migration (Flyway), que você vai usar com Spring.

☐ **S37. Treino de volume**

30 problemas de SQL no LeetCode (Database) ou HackerRank, sendo pelo menos 10 Medium. Registre no README o link de cada um.

☐ **S38. Desafio: 10 relatórios de negócio**

Cada um numa query só: faturamento por categoria no trimestre; ticket médio por cliente; clientes inativos há 90 dias; produtos com estoque abaixo de 5 que venderam no mês; taxa de cancelamento por mês; melhor dia da semana em vendas; top 3 produtos por categoria (window function); crescimento mês a mês em %; clientes que compraram só uma vez; pedidos com mais de 3 itens diferentes.

Atenção: Critério: escrever todas sem consultar nada em menos de 2 horas.

E1-E4 | Treino de entrevista **NOVO**

Contínuo a partir do módulo de Strings. Sem IA, com cronômetro.

☐ **E1. Um problema por dia**

A partir do módulo de Strings, 1 problema Easy por dia (LeetCode ou HackerRank) em Java. Depois de Collections, 2 Easy + 1 Medium por semana. Meta antes do Spring: 60 resolvidos.

Atenção: Limite: 30 min por Easy. Estourou? Veja a solução, feche, e reescreva do zero no dia seguinte.

☐ **E2. Explicar em voz alta**

Toda semana, grave 5 minutos explicando um conceito sem ler nada: equals/hashCode, checked vs unchecked, List vs Set vs Map, passagem por valor, INNER vs LEFT JOIN, o que é índice.

Atenção: Se travou, você não sabe. Volta e estuda.

☐ **E3. Perguntas técnicas**

Responda por escrito, com suas palavras e um exemplo de código: diferença entre == e equals; por que String é imutável; interface vs classe abstrata; o que é polimorfismo; o que acontece num HashMap com colisão; o que é uma transação; o que é REST; o que é injeção de dependência.

☐ **E4. Live coding simulado**

Uma vez por semana, resolva um exercício desta lista falando em voz alta, com a câmera ligada ou para alguém, sem IDE com autocomplete (use um editor simples).

Atenção: Entrevista de júnior costuma ter isso. Autocomplete não estará lá para salvar.

Critério para começar o Spring

Todos marcados, com honestidade. Um só em aberto = ainda não é hora.

- ☐ Resolvo os exercícios de Strings e Collections desta lista sem IA, sem consultar e dentro do tempo.
- ☐ Sei explicar sem ler: == vs equals, Integer cache, divisão inteira, por que double não serve para dinheiro.
- ☐ Sei explicar equals/hashCode, como o HashMap funciona e o custo de cada coleção.
- ☐ Sei explicar checked vs unchecked e modelo exceções de negócio com mensagem útil.
- ☐ Crio projeto Maven do zero, rodo mvn test pelo terminal e leio stack trace sem pânico.
- ☐ Escrevo teste antes do código (TDD) nos exercícios de regra de negócio.
- ☐ Testo um service com Mockito, incluindo os caminhos de erro.
- ☐ Escrevo JOIN de 3 tabelas, GROUP BY com HAVING e window function sem consultar.
- ☐ Faço CRUD com JDBC usando PreparedStatement e transação com rollback.
- ☐ Sei os status HTTP de cor e desenho uma API REST sem verbo na URL.
- ☐ Tenho 60 problemas resolvidos e já fiz live coding simulado pelo menos 4 vezes.
- ☐ O projeto final roda em outra máquina com git clone + docker compose up + mvn test.